



ANKARA YILDIRIM BEYAZIT UNIVERSITY

**FACULTY OF ENGINEERING AND NATURAL
SCIENCES DEPARTMENT OF COMPUTER
ENGINEERING**

GRADUATION PROJECT FINAL REPORT

1. General Information

Group Members (Names and ID numbers):	Zeynep Üstün (21050111048), Merve Çankaya (21050111052), Zeki Furkan Yıldız (21050111019)
Research Title:	Smart Municipality Application: Artificial Intelligence-Supported Environmental and Infrastructure Notification System
Supervisor Name:	Doç. Dr. Muhammed Abdullah BÜLBÜL

2. Abstract

This comprehensive graduation project introduces an autonomous, secure, and end-to-end "Smart Municipality Application" designed to modernize urban infrastructure maintenance, eliminate operational communication bottlenecks, and optimize local government resource allocation. Traditional municipal infrastructure reporting systems rely heavily on manual civil complaints or periodic physical patrols. These conventional methods are inherently reactive, prone to human subjectivity, costly to maintain, and cause significant delays in repairing critical public hazards.

To resolve these architectural limitations, we have engineered a decentralized, low-cost, and scalable infrastructure leveraging a cross-platform mobile client built with Flutter and Dart, a robust server-side backend designed with Node.js, Express, and TypeScript utilizing a rigid Layered Architecture, and an optimized, on-device Edge AI deep learning engine for real-time computer vision inference.

The core intelligence of the client-side infrastructure features an extensively optimized, lightweight convolutional object detection model based on the State-of-the-Art (SOTA) YOLOv8 Nano architecture. Trained on a diverse structural dataset, the model is engineered to detect eight distinct categories of critical urban anomalies—including potholes, overflowing public garbage bins, unmapped open electrical panels, and fallen or damaged trees—locally on the edge device. By compiling the native model weights into a serialized TensorFlow Lite (.tflite) format, inference is executed entirely offline directly within the mobile application runtime environment. This edge approach achieves an ultra-low inference latency of approximately 45 milliseconds per frame while delivering an exceptional mean Average Precision of 97.5% at an intersection-over-union threshold of 0.5 (mAP₅₀). To maximize the operational trade-off between Precision and Recall under diverse ambient lighting conditions, the confidence threshold was systematically optimized and locked at 40% (0.40). This threshold acts as an optimal operational sweet spot; it guarantees that ambiguous anomalies are successfully captured while remaining strictly high enough to prevent excessive false-positive reports from overloading municipal data tables.

On the server-side, the backend architecture prioritizes strict type safety, modularity, and high-throughput transaction isolation. Built on TypeScript to ensure compile-time error checking, the enterprise-grade API separates concerns cleanly across dedicated route, data transfer object (DTO), controller, service, repository, and policy layers. Security at the gateway is enforced via a hybrid validation strategy. Stateless user and worker authentication is guaranteed through JSON Web Tokens (JWT) processed via secure

middleware, while the integrity of incoming structural payload parameters is strictly validated at the runtime boundary using schemas defined by the Zod validation library. Furthermore, the ingestion of unstructured binary media assets is isolated via Multer middleware, which writes high-definition images to local server disk storage as static URIs, while lightweight relational metadata is asynchronously persisted to a Cloud Firestore database.

Experimental field validations and comparative hardware testing confirm that this hybrid architectural synthesis significantly outperforms traditional centralized cloud-inference solutions. By moving the deep learning compute layer to the edge, the system completely removes linear cloud GPU hosting expenditures, protects user privacy under data protection acts by instantly discarding non-anomalous background imagery at the device level, and remains operational in cellular dead zones. The resulting platform successfully bridges the communication gap between citizens and local authorities, providing a scalable, low-latency framework for next-generation smart city automation and predictive urban asset management.

3. Introduction

3.1. Problem Definition

Urban centers globally are facing unprecedented growth, which directly increases the strain on municipal infrastructure. Incidents such as potholes, damaged utilities, overflowing public waste bins, and fallen tree branches during extreme weather events require immediate intervention to ensure public safety and maintain quality of life. However, contemporary urban administration workflows suffer from major communication bottlenecks.

Currently, reporting mechanisms are sluggish and reactive. Infrastructure defects are either discovered through routine, manual municipal patrols—which are labor-intensive, costly, and intermittent—or through citizen calls that often lack precise localization or objective descriptions. Because citizens are disconnected from a centralized, transparent reporting process, minor anomalies frequently escalate into severe structural damage or public hazards before local authorities can intervene. This structural delay leads to suboptimal workforce planning, inefficient fleet management, and a severe escalation in maintenance costs for local governments.

3.2. Project Objectives

The objective of this project is to build a low-cost, secure, and end-to-end autonomous infrastructure notification network that eliminates the structural delays inherent in traditional municipality pipelines. The specific objectives are defined as follows:

- **Transforming Mobile Hardware into Smart Sensors:** To democratize urban maintenance by allowing everyday citizens to capture public infrastructure anomalies using standard smartphone cameras, converting raw imagery into verified spatial data tokens automatically.
- **Decentralized Edge AI Implementation:** To run deep learning inference directly on the edge hardware (the mobile client) via an optimized lightweight object detection

architecture. By operating fully offline and on-device, the system eliminates the recurring costs of cloud-hosted GPU compute clusters, reduces server dependency to during real-time inference, and provides immediate feedback with zero network latency.

- **Unified Automation and Resource Optimization:** To implement a secure cross-platform infrastructure that synchronizes citizen reports with real-time field worker and administrative dashboards. This automation enables dynamic routing for municipal maintenance teams, shifting local government operations from a reactive posture to a data-driven proactive strategy via autonomous task assignment.
- **Data Privacy and Zero-Trust Security:** To establish a privacy-by-design framework that aligns with personal data protection regulations. By processing visual data on the edge and instantly discarding non-anomalous background imagery, combined with a stateless JWT authentication and rigid Zod payload validation at the backend gateway, the system ensures enterprise-grade isolation against unauthorized access and data breaches.

4. Literature Review

Traditional civic complaint platforms, such as national or local citizen hotlines (e.g., CİMER or local municipal "Beyaz Masa" portals in Turkey), rely entirely on manual human input. On these standard portals, users must select categories from complex nested menus, write subjective textual explanations, and upload massive, uncompressed images. These inputs must then be manually filtered, verified, and assigned by municipal personnel. This dependency on subjective human reporting introduces high operational overhead and leads to a high frequency of misclassified reports.

In recent academic literature, computer vision architectures have been extensively proposed to automate infrastructure monitoring. However, mainstream research tends to prioritize centralized cloud-inference frameworks. In these architectures, mobile devices act merely as data collectors that transmit uncompressed high-definition video frames or photos over mobile networks (4G/5G) to powerful remote servers hosting large-scale convolutional networks (CNNs) or vision transformers.

While cloud computing provides substantial raw processing power, it introduces critical structural drawbacks that make it impractical for widespread municipal application:

1. **Bandwidth and Cloud Hosting Costs:** Continuously streaming media files strains network bandwidth and incurs expensive cloud storage and ongoing GPU compute hosting fees.
2. **Privacy Concerns:** Uploading unfiltered citizen photos to cloud servers introduces liability regarding General Data Protection Regulations (GDPR/KVKK), as images might contain identifiable human faces or vehicle license plates.
3. **Network Dependency:** Cloud systems fail completely in underground tunnels, remote areas, or situations where cellular connections are weak or overloaded.

Our Core Difference and Contribution: Our project addresses these architectural limitations by implementing an On-Device Edge AI (TFLite) architecture integrated with a hybrid, secure enterprise backend. By executing model inference locally within the client

application, the system processes image grids at the edge. The deep learning model identifies, crops, and validates anomalies in real time without ever sending raw, unverified data streams across the network. This approach guarantees strict data privacy, as non-anomalous imagery is destroyed instantly at the device level. Furthermore, it allows for seamless offline operations and scales across thousands of concurrent mobile clients without demanding a linear increase in server compute budgets, completely eliminating cloud-based GPU operational costs.

5. Methods, Tools, and Technologies

This section details the hybrid system architecture, which integrates an on-device Edge AI module, a secure server-side infrastructure, and a real-time cross-platform mobile client. The methodology is structured around achieving low latency, high throughput, and strict data privacy.

5.1. Artificial Intelligence Core: YOLOv8n

The core visual recognition engine of the system is built upon the YOLOv8 Nano (YOLOv8n) architecture, selected specifically for its optimal speed-accuracy trade-off in mobile environments.

Architectural Selection & Parameter Efficiency: Unlike medium or large-scale vision models, YOLOv8n features a highly optimized, lightweight structure with approximately 3.2 million parameters. This minimal computational footprint is a strict necessity; it prevents thermal throttling and CPU/NPU memory overload on standard mobile devices during continuous real-time video processing. Its anchor-free architecture allows for dynamic spatial recognition, effectively detecting irregularly shaped urban anomalies without relying on rigid bounding box presets.

Hyperparameter Optimization and Confidence Thresholding: In urban hazard management, the operational cost of missing a critical hazard significantly outweighs the cost of verifying a false alarm. To maximize this operational trade-off, the model's confidence threshold was strategically locked at 0.40. This configuration intentionally prioritizes Recall—ensuring the system is highly sensitive to potential public threats. Simultaneously, locking the threshold specifically at 0.40 (rather than lower values) prevents extreme drops in Precision, successfully mitigating the risk of Alarm Fatigue for field workers that would occur if the model were too sensitive.

Data Augmentation Strategy: To prevent model overfitting and ensure maximum generalization under unpredictable real-world urban conditions (shadows, varying angles, partial occlusions), an aggressive data augmentation pipeline was configured during the training phase. The precise hyperparameters applied to the dataset included: Mosaic (1.0) and Mixup (0.3) to enhance contextual and structural learning by blending multiple images; Random Erasing (0.4) to simulate partial occlusions of urban hazards; Perspective (0.001), Degrees (15.0), Translate (0.1), and Scale (0.5) to actively account for the varying camera angles and distances typical of mobile smartphone photography; and HSV Value manipulation (0.6) along with Horizontal Flip (0.5) to stabilize the model's predictive capabilities across diverse ambient lighting conditions.

Edge Deployment (Training vs. Runtime Conversion): While the model was trained using the native PyTorch framework, deploying a standard PyTorch weights file directly on a mobile device is computationally prohibitive. To resolve this architectural bottleneck, the finalized weights from the optimal training run were quantized and serialized into the TensorFlow Lite (.tflite) format. This critical cross-platform conversion is what strictly allows the inference process to be decoupled from cloud servers, executing entirely offline and natively within the Flutter runtime environment with an ultra-low latency of approximately 45 milliseconds per frame.

5.2. Backend Architecture: Node.js, Express & TypeScript

The server-side infrastructure functions as a Zero-Trust gateway, engineered to handle high-throughput concurrent requests without blocking execution threads.

- **TypeScript and Type Safety:** The entire backend was developed using TypeScript. By enforcing strict type safety and compile-time error checking, potential runtime crashes caused by undefined data structures were eliminated, maintaining a stable and resilient platform.
- **Rigid Layered Architecture (Separation of Concerns):** To maximize modularity and maintainability, the backend enforces a strict layered design:
 - **Routes Layer:** Defines API endpoints and intercepts incoming HTTP traffic.
 - **DTO (Data Transfer Object) Layer:** Specifies the exact shape of incoming and outgoing data payloads.
 - **Controllers:** Manages HTTP request/response cycles without containing business logic.
 - **Services:** Houses the core algorithmic and operational business rules.
 - **Repositories:** Abstracts database interactions, decoupling the business logic from the underlying Firestore SDK.
 - **Policies / Middlewares:** Executes pre-flight security checks, authentication, and error handling.
- **Security and Payload Validation (JWT & Zod):** Security is implemented via a dual-layered gateway mechanism. First, JSON Web Tokens (JWT) provide stateless user authentication, verifying the identity and role (Citizen/Worker/Admin) of the client without consuming server memory. Second, the Zod validation library acts as a rigid runtime boundary; it strictly evaluates the structural integrity of incoming JSON payloads against predefined DTO schemas, instantly rejecting malformed requests before they reach the database.
- **Hybrid Storage Strategy:** To optimize cloud billing and query performance, a hybrid data persistence strategy was adopted. High-definition binary assets (images) are intercepted via Multer middleware and stored locally on the server's physical disk. Simultaneously, lightweight relational metadata (coordinates, anomaly type, timestamp, and the static image URI) is asynchronously written to a Cloud Firestore NoSQL database.

5.3. Frontend & Mobile: Flutter & Dart

The client interface was developed utilizing the Flutter framework, providing a unified, high-performance user experience across diverse mobile ecosystems.

- **Single Codebase and Role-Based User Interfaces:** Utilizing Dart, the platform leverages a single codebase to serve multiple stakeholders. The application architecture dynamically routes interfaces based on the injected JWT claims, presenting distinct, role-specific environments:
 - **Citizen Interface:** A comprehensive civic engagement portal. Rather than displaying raw AI bounding boxes, the deep learning inference runs silently in the background during report submission, ensuring a frictionless user experience. Beyond reporting anomalies, citizens can track their complaint history, browse municipal events and announcements, receive real-time weather updates, and access an emergency button.
 - **Worker Dashboard (Municipal Employee):** A specialized task management and dispatch environment. Workers can monitor active workflows, review emergency complaints, correct misclassified issue types directly from the field, and manage team assignments. For each complaint, workers can access the reported location, uploaded images, and detailed descriptions provided by citizens, enabling faster and more accurate field operations. The universal emergency module is also integrated into the interface to support field safety and rapid response.
 - **Admin Overview:** A strategic monitoring and analytics dashboard. Administrators have full visibility over all complaint lifecycles and can apply complex filters to the incident map. Crucially, the dashboard provides institutional analytics, visualizing complaint densities and resolution rates across specific urban regions and municipal departments, thereby allowing local governments to optimize their resource allocation.
- **Asynchronous State Management and UI Fluidity:** Client-side state is managed asynchronously to ensure complete UI fluidity. When a user captures or uploads an image, the heavy TFLite object detection operation is executed on a background isolate. This protects the main UI thread, preventing the application from freezing or lagging during the image validation and processing phase.
- **Real-time Synchronization (Firestore Streams):** The worker and admin dashboards integrate seamlessly with Firestore's native real-time listeners. When a validated hazard is persisted to the database by the Node.js server, the Flutter client receives immediate delta updates (fetching only the modified data rather than the entire database) via persistent cloud connections. This stream architecture automatically refreshes the workers' active task feeds and incident maps without requiring manual reloads, achieving true autonomous workflow synchronization.

6. Implementation & System Architecture

This section delineates the end-to-end execution lifecycle of the Smart Municipality Application. The operational flow, from the initial user interaction to the final task resolution by municipal workers, is governed by a strict sequence of edge computing, secure API validation, and real-time database synchronization. The complete system architecture flowchart is illustrated below.

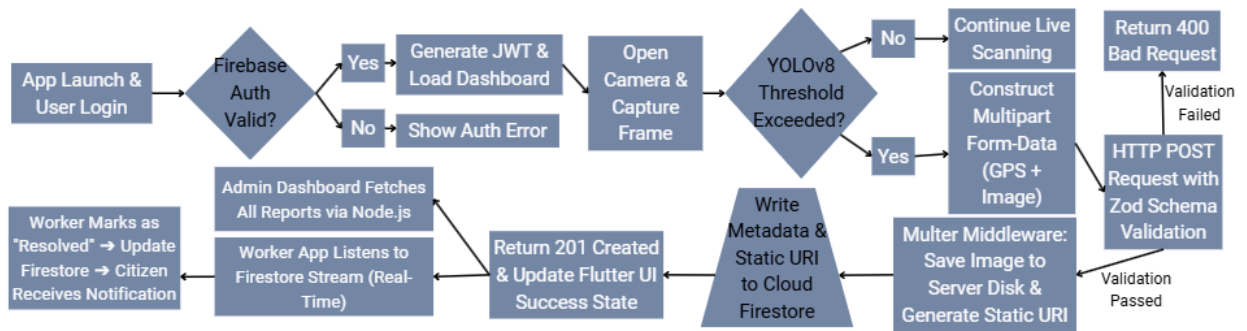


Figure 1: End-to-end system architecture and operational data flow, illustrating the lifecycle of an anomaly report from edge AI detection to backend Firestore persistence.

The architectural workflow is executed through the following sequential phases:

Step 1: Application Launch and Secure Authentication

The operational lifecycle initiates when a user launches the Flutter application. The system immediately intercepts the login request to verify credentials via Firebase Authentication.

- **Validation Failure:** If the credentials are invalid or missing, the system halts the progression and returns an Authentication Error to the user interface.
- **Validation Success:** Upon successful authentication, the backend generates a secure JSON Web Token (JWT) encapsulating the user's role constraints. The application then dynamically loads the corresponding role-based environment (Citizen, Worker, or Admin Dashboard).

Step 2: Edge AI Capture and Anomaly Classification

Within the Citizen interface, the user opens the camera module and captures a photographic frame of a suspected urban hazard. This frame is instantly processed by the YOLOv8n TFLite model directly on the device.

- **Threshold Exceeded (Automatic Detection):** If the model successfully detects an anomaly, the system autonomously classifies the hazard type (e.g., pothole, damaged electrical panel) based on the AI's prediction, streamlining the reporting process for the user.
- **Threshold Not Exceeded (Manual Override):** If the AI fails to automatically detect the object due to extreme lighting conditions, occlusion, poor image quality, or low confidence scores, the reporting workflow remains uninterrupted. The citizen may either upload a new photograph to trigger another detection attempt or manually select the appropriate anomaly category through the user interface. This robust fallback mechanism ensures that no civic issue goes unreported due to temporary image quality problems or algorithmic limitations.

Step 3: Payload Construction and Rigid Gateway Validation

Once the hazard is verified by the Edge AI, the mobile client constructs a Multipart/Form-Data package. This payload securely encapsulates the cropped anomaly image alongside vital relational metadata (precise GPS coordinates, timestamp, and anomaly classification). The package is then transmitted to the Node.js server via an HTTP POST request. At the backend gateway, the payload is intercepted by the Zod validation middleware, which strictly evaluates the structural integrity of the incoming data against predefined schemas.

- **Validation Failed:** If the payload is malformed (e.g., missing spatial data or incorrect data types), the gateway instantly rejects the transmission, returning an HTTP 400 Bad Request to prevent database corruption.
- **Validation Passed:** If the schema validation is successful, the payload is authorized to enter the storage pipeline.

Step 4: Hybrid Persistence and URI Generation

To optimize both performance and cloud billing, the system executes a hybrid storage strategy. The validated request is processed by the Multer middleware, which extracts the high-definition binary image and writes it securely to the server's local disk, generating a static URI. Simultaneously, the lightweight metadata (GPS, category, and the newly generated static URI) is asynchronously transmitted and persisted as a document within the Cloud Firestore database.

Step 5: Client Confirmation and Intelligent Routing Recommendation

Following successful persistence, the Node.js server responds to the mobile client with an HTTP 201 Created status code, triggering a success state update on the citizen's Flutter UI. Simultaneously, the system processes the verified data to assist municipal operations through a decision-support mechanism:

- **Contextual Institution Recommendation:** Based on the classified anomaly category (whether detected by the AI or selected manually), the backend dynamically determines the most appropriate municipal department or external utility company. When the report drops into the municipal dashboard, this specific department is highlighted as the primary.
- **Human-in-the-Loop Assignment:** To prevent misallocations and ensure operational flexibility, the system does not force an autonomous dispatch. Instead, municipal dispatchers or administrators review the incoming report. They are granted the authorization to either approve the recommended institution or manually reassign the task to a different department before the task is officially dispatched to the field teams.
- **Real-time Worker Synchronization:** Once the task is officially assigned by the administrator, the authorized field workers receive an immediate delta update via Firestore Streams. This persistent connection automatically updates the workers' active task feeds and incident maps, ensuring rapid response times while maintaining strict human oversight.

Step 6: Task Resolution and Citizen Feedback Loop

The lifecycle concludes when a municipal worker physically navigates to the GPS coordinates and resolves the urban issue. The worker updates the task status to "Resolved" directly through their mobile dashboard. This action triggers a final Firestore update, which automatically dispatches a push notification to the original reporting citizen, successfully closing the civic feedback loop and ensuring transparent municipal operations.

7. Presentation of Findings & Outputs

This section visualizes the empirical results of the model and analyzes the real-world operational outputs of the Smart Municipality Application across various edge environments.

7.1. Training Convergence and Validation Metrics

The YOLOv8 Nano architecture demonstrated exceptional stability during the training phase. Across 100 epochs, the model's convergence was highly consistent, ultimately achieving a peak mAP₅₀ score of 97.5% on the custom urban anomaly dataset. Crucially, the model maintained a realistic mAP₅₀₋₉₅ score of ~71%, proving that it did not simply memorize the dataset (overfitting) but successfully generalized spatial variations to detect anomalies under complex, real-world conditions.

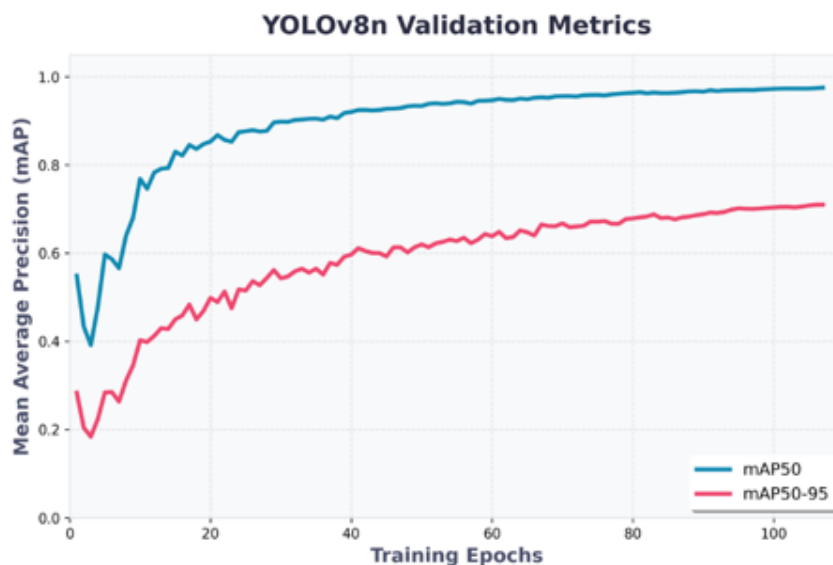


Figure 2: YOLOv8n training convergence and mean Average Precision (mAP) stability metrics evaluated across 100 epochs on the custom urban hazard dataset.

7.2. Multi-Class Inference Output and Field Performance

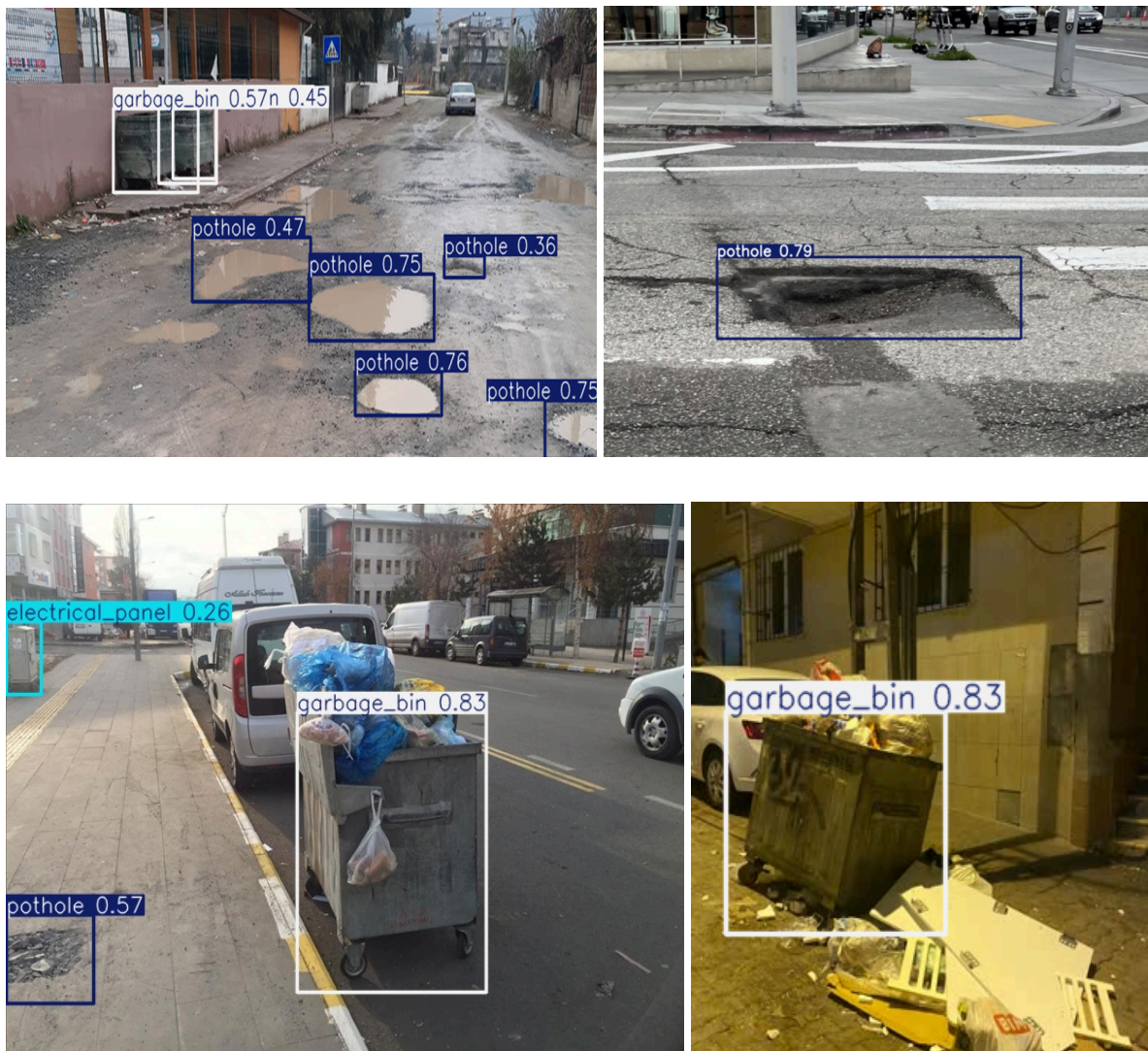
The model was trained to autonomously classify eight distinct critical urban hazards: Pothole, Garbage Bin (Overflow), Electrical Panel (Open/Hazard), Bench (Damaged), Scooter (Improper Parking), Traffic Light (Malfunction), Illegal Poster, and Tree (Fallen).

During field testing, the AI consistently exhibited robust detection capabilities across all classes, though confidence distributions varied based on object morphology. For structurally

rigid anomalies with distinct geometric boundaries (such as Electrical Panels, Benches), the model frequently achieved peak confidence scores, often exceeding the 80-90% range.

Conversely, for dynamic and topologically irregular hazards like Potholes and Trees, the confidence scores naturally displayed a wider variance. Depending heavily on the camera angle, capture distance, and ambient lighting, these irregular objects yielded a broader spectrum of confidence levels. However, even in highly variable scenarios, the model successfully captured multiple instances within a single frame, reliably maintaining a strong margin above the 40% operational threshold.

Occasional degradations in optimal performance and confidence drops were primarily observed under specific edge-case conditions, such as extreme sun glare, heavy shadowing over the anomaly, or partial occlusion by vehicles and pedestrians. Despite these environmental constraints, the model's overall generalization remained highly stable for autonomous field reporting.



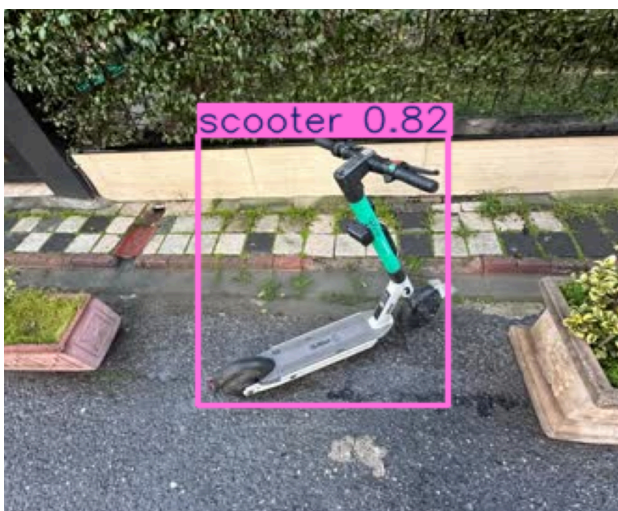
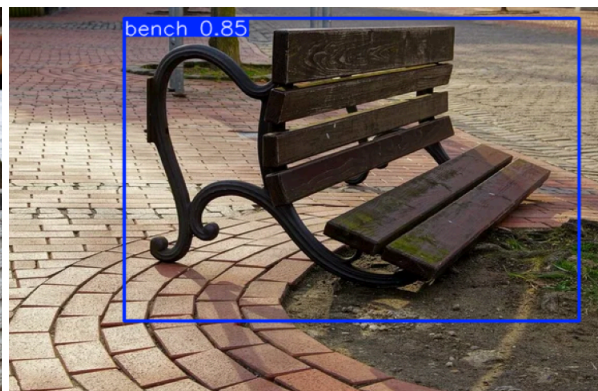




Figure 3: Real-world inference outputs of the YOLOv8n edge model, successfully detecting and classifying diverse urban anomalies (e.g., potholes, overflowing garbage bins) under varying ambient lighting and morphological conditions.

7.3. Error Analysis and Edge Cases: The "Fallen Tree" Challenge

While the model demonstrated high accuracy across structured anomalies, empirical tests revealed a specific struggle with the *Fallen Tree* class. This is not due to dataset imbalance, but rather strict computer vision constraints:

- **High Morphological Variation:** Unlike electrical panels or benches, which possess predictable geometric boundaries, fallen trees lack a standardized physical structure.

The extreme variance in branch patterns, leaf density, and trunk angles makes it difficult for the bounding box regressor to encapsulate the entire object accurately.

- **Camouflage Effect and Low Contrast:** Fallen trees often blend seamlessly into their natural surroundings (grass, dirt, or other foliage). This lack of sharp pixel contrast at the object boundaries causes the convolutional layers to lose distinct edge features, occasionally resulting in missed detections or lower confidence scores.

7.4. Architectural Shift: PyTorch to TFLite Quantization and Edge Hardware Variability

A critical engineering observation was made during the deployment pipeline regarding inference consistency. While the original PyTorch weights (best.pt) executed on the PC environment yielded highly stable detections, compiling the model into the .tflite format for edge execution introduced occasional detection fluctuations across varying devices (e.g., physical tablets vs. emulators).

This discrepancy is not a flaw in the model's training, but rather a direct consequence of two core edge computing factors:

- **Mathematical Quantization Loss:** To enable ultra-low latency inference on mobile processors, the TFLite conversion inherently compresses the model's complex mathematical precision. By converting high-precision 32-bit floating-point tensors into lower-precision formats, continuous weight spaces are truncated. In deep convolutional layers, these micro-rounding errors accumulate, which can slightly alter the final feature map and cause minor variations in the raw confidence outputs of the bounding box regressor.
- **Hardware-Specific Delegate Execution:** Inference on edge devices is highly dependent on the underlying hardware architecture. An Android emulator relies on CPU virtualization, whereas a physical mobile device leverages advanced hardware acceleration components such as Neural Processing Units or GPUs via the Neural Networks API. Because different chip architectures handle low-precision matrix multiplications and activation functions uniquely, the exact same .tflite model can produce slightly different prediction scores depending on the device executing it.

Consequently, rather than a fixed numerical drop, this combination of mathematical truncation and hardware variability means that highly ambiguous or borderline anomalies might occasionally fluctuate just below the strict system threshold, depending entirely on the specific mobile hardware capturing and processing the frame.

7.5. User Interface (UI) Outputs & Role-Based Workflows

As discussed in the system architecture, the mobile client was developed using a single Flutter codebase that dynamically routes the user experience based on secure JWT role claims. The resulting user interfaces are highly optimized for fluid navigation and asynchronous data fetching. The functional outputs of these role-based environments are detailed below:

7.5.1. The Citizen Portal: Frictionless Civic Engagement

The citizen-facing mobile application is designed as a comprehensive smart city platform that encourages public participation while minimizing user effort and technical complexity.

Centralized Home Dashboard: In addition to the primary “Report an Issue” functionality, the home dashboard provides citizens with access to localized weather information, municipal announcements (such as tax payment reminders), and quick links to emergency services (e.g., City 153/SOS). This centralized design aims to improve accessibility to frequently used civic services.

AI-Assisted Reporting Interface: The reporting interface streamlines the complaint submission process by automating several tasks. When a user captures an image, the application automatically retrieves the device’s GPS coordinates and attaches them to the report. Furthermore, a TensorFlow Lite version of the YOLOv8n object detection model performs on-device inference and suggests the most relevant issue category (e.g., “Traffic/Signs”). This reduces the need for users to manually navigate category menus and helps standardize report classification.

Transparent Complaint History: Citizens can monitor the progress of their submissions through the “My Complaint History” section. Report statuses such as New, In Progress, and Resolved are synchronized with the backend system in real time, providing transparency and establishing a continuous feedback mechanism between citizens and municipal authorities.

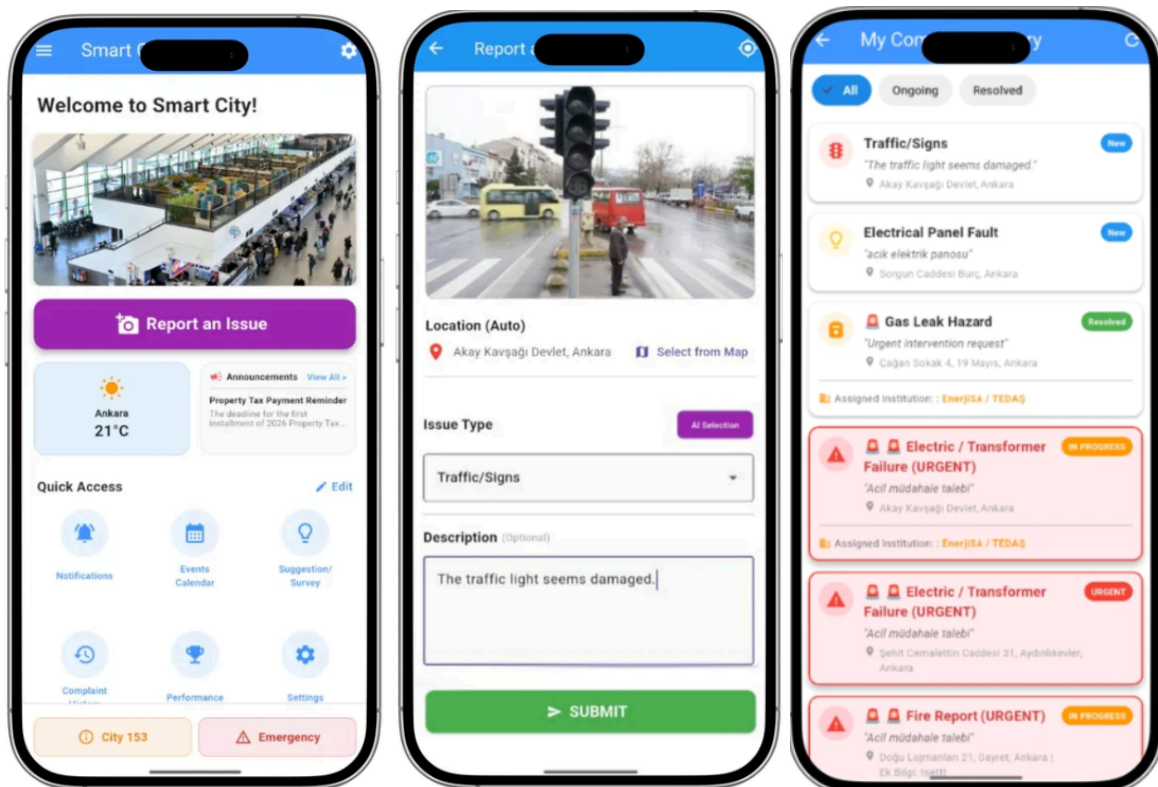


Figure 4: The Citizen Portal mobile interface, featuring the centralized civic dashboard, AI-assisted anomaly reporting module, and real-time transparent complaint history tracking.

7.5.2. Field Worker Dashboard: Autonomous Task Management

The interface designed for municipal field workers strictly prioritizes operational efficiency, rapid response times, and robust human oversight.

- **Real-Time Synchronization:** Unlike the citizen view, the worker dashboard actively listens to persistent Firestore Streams. Incoming reports are instantly populated in the task feed with distinct visual hierarchies and status indicators, requiring no manual page reloads.
- **Human-in-the-Loop Validation & Intelligent Routing:** To prevent operational misallocations caused by user error or initial AI misclassification, the dashboard features a dynamic validation mechanism. Field workers or supervisors possess the authorization to review and manually modify the anomaly category. Based on this finalized category, the system's backend immediately provides an "Intelligent Routing Recommendation," automatically highlighting the most relevant municipal department or external utility (e.g., Parks & Gardens Dept., or Regional Energy Distributors) as the primary assigned institution, streamlining the triage process.
- **Priority Toggling and Dispatching:** Workers can filter tasks by specific institutions or toggle the "Show Urgent Complaints Only" switch to instantly isolate critical, high-priority hazards. The interface features direct action buttons (such as "ASSIGN TEAM"), allowing dispatchers to allocate human resources dynamically directly from the mobile edge, effectively minimizing bureaucratic delays while maintaining strict operational control.

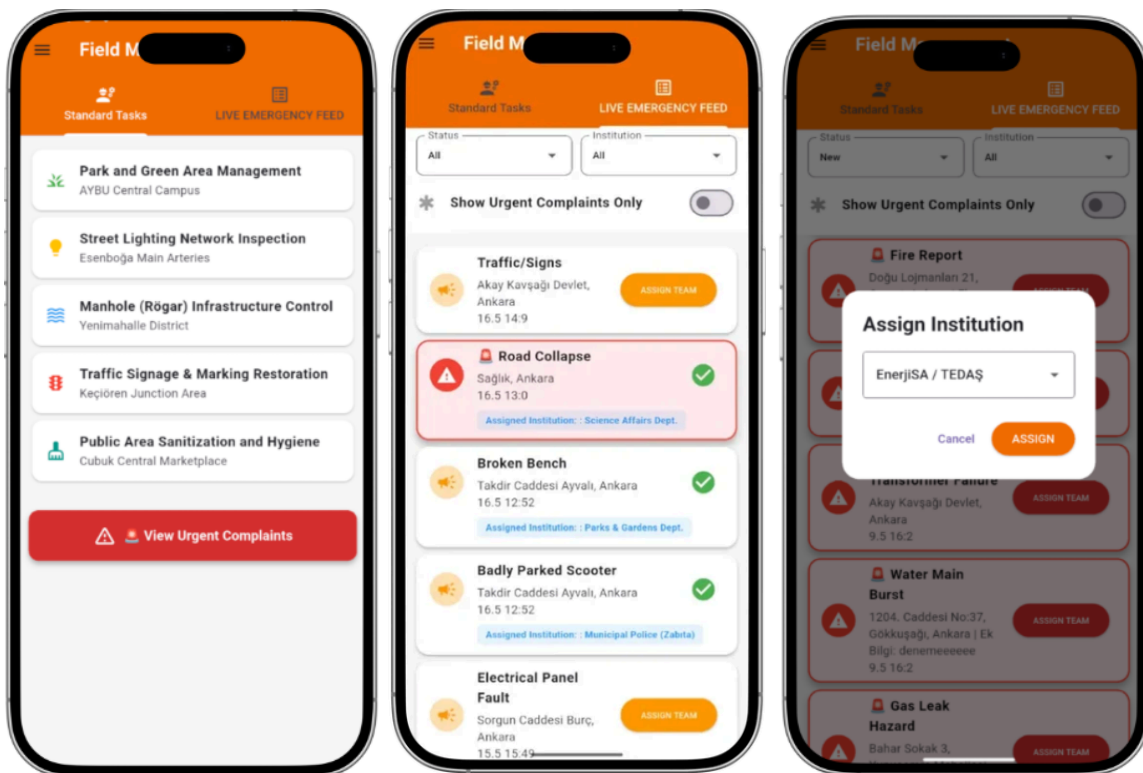


Figure 5: The Field Worker mobile dashboard, illustrating the real-time task management feed synchronized via Firestore Streams, urgent complaint isolation, and dynamic institutional dispatching modules.

7.5.3. Central Monitoring and Analytics System

The administrative interface provides municipal authorities with a centralized platform for monitoring citizen reports and analyzing city-wide complaint trends. Rather than directly managing operational workflows, the system focuses on providing situational awareness through real-time data visualization and analytical insights.

Comprehensive Complaint Monitoring: Administrators can access and review all submitted reports through a unified dashboard. Each report is displayed together with its location, category, submission time, and current status. The interface also highlights urgent incidents identified by the system, enabling authorities to quickly recognize critical situations while maintaining visibility over all incoming complaints.

Regional and Institutional Analytics: The platform aggregates complaint data to generate regional statistics and spatial visualizations, allowing administrators to identify areas with high complaint densities and recurring urban issues. In addition, analytical dashboards display the distribution of reports across responsible institutions and service categories. These insights provide a city-wide overview of complaint patterns and support data-driven planning and resource assessment.

Real-Time Status Tracking: The system continuously synchronizes report statuses, enabling administrators to monitor the progression of complaints from submission to resolution. This functionality provides a transparent overview of ongoing municipal activities and helps authorities evaluate service responsiveness across different regions and institutions.

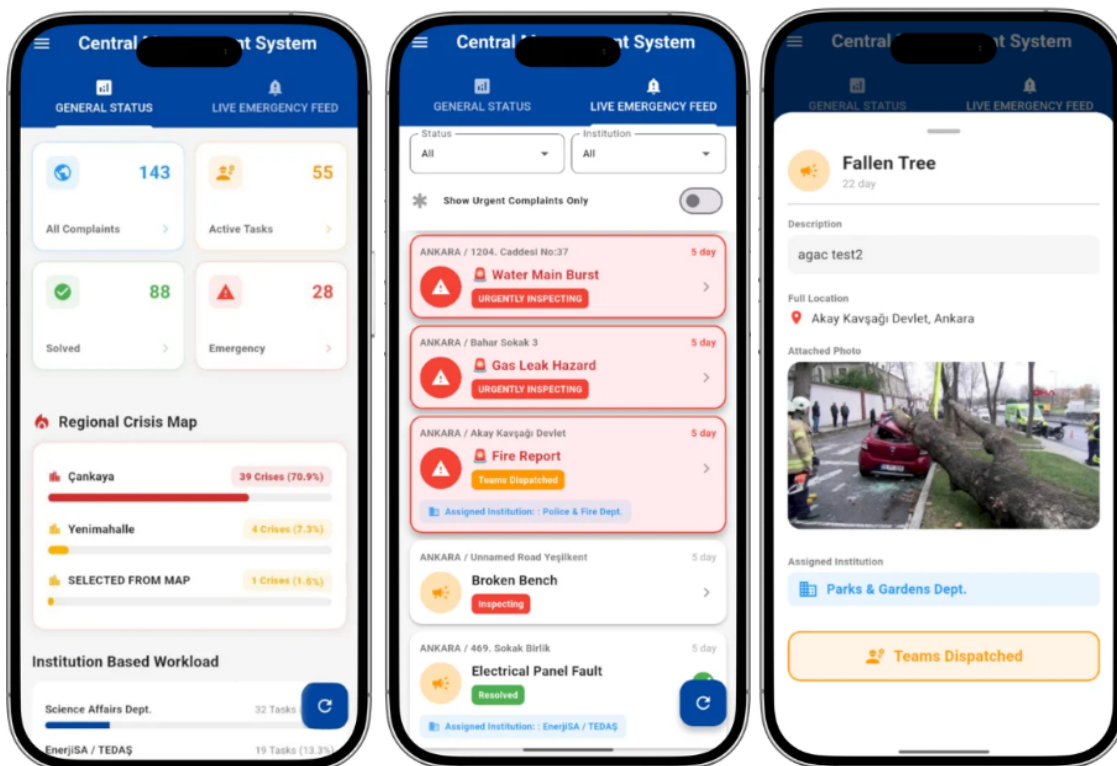


Figure 6: The Central Management System designed for municipal administrators, displaying the real-time regional crisis map, institutional workload analytics, and the live emergency feed.

8. Project Schedule & Gantt Chart

8.1. Project Schedule Status Table

Phase	Activity / Work Package	Timeline	Status	Completed Milestone Notes
P1	Research & Requirement Analysis	Sept - Oct 2025	COMPLETED	Literature review and system architecture requirements finalized.
P2	UI/UX Design & Prototyping	Oct - Nov 2025	COMPLETED	High-fidelity prototypes for Citizen, Worker, and Admin views designed.
P3	Mobile Frontend (Flutter)	Nov 2025 - Jan 2026	COMPLETED	Core application screens and multilingual UI logic implemented.
P4	AI Model Training & Conversion	Nov 2025 - Jan 2026	COMPLETED	YOLOv8n model trained and successfully converted to .tflite format.
P5	Edge AI & Flutter Integration	Dec 2025 - Jan 2026	COMPLETED	On-device offline inference pipeline embedded into Flutter bundle.

P6	Backend Infrastructure & API	Dec 2025 - Feb 2026	COMPLETED	Node.js (TypeScript) API built with JWT auth and Zod schemas.
P7	System Integration & Testing	Feb - Mar 2026	COMPLETED	End-to-end connectivity (Camera>AI>Backend>Firestore) verified.
P8	System Validation & Reporting	Apr - May 2026	COMPLETED	Performance metrics compiled, poster designed, and graduation defense completed.

8.2. Final Project Gantt Chart

Phase / Activity	Sept '25	Oct '25	Nov '25	Dec '25	Jan '26	Feb '26	Mar '26	Apr '26	May '26
P1: Research & Planning									
P2: UI/UX Design									
P3: Mobile Frontend									

P4: AI Model Training			 	 	 				
P5: Edge AI Integration				 	 				
P6: Backend Infrastructure				 	 	 			
P7: Integration Testing						 	 		
P8: Reporting & Defense								 	 

9. Challenges Encountered & Resolution

During the development process, specifically during system integration, edge AI optimization, and dynamic data management, the engineering team encountered several architectural bottlenecks. These challenges were systematically addressed using the following engineering solutions:

9.1. Artificial Intelligence & Performance Bottlenecks

- **Challenge 1: Maintaining UI Responsiveness During On-Device Image Validation**
 - **Problem:** Mobile applications rely on a single main UI thread to render frames and process user interactions. Executing computationally intensive tasks such as image preprocessing, object detection, and postprocessing directly alongside UI operations can interrupt the rendering cycle, resulting in frame drops, temporary freezes, and a degraded user experience. Additionally, the anomaly detection system needed to remain functional even in areas with limited or no internet connectivity.
 - **Solution:** This challenge was addressed through a multi-layered optimization strategy. First, the anomaly detection model was implemented using the lightweight YOLOv8 Nano architecture and deployed in the highly optimized TensorFlow Lite (TFLite) format, significantly reducing computational

overhead and memory consumption. Second, the image analysis pipeline was executed within Dart Isolates, separating heavy processing tasks from the main UI thread and ensuring smooth user interactions without screen jank. Finally, because the TFLite model is stored and executed directly on the device, anomaly detection can operate entirely offline, eliminating network dependency while improving response time, reliability, and user privacy.

- **Challenge 2: Algorithmic Instability Under Mobile Camera Jitter**

- **Problem:** During field tests, motion blur from shaky smartphone cameras and varying ambient lighting caused the AI model to miss critical hazards, increasing False Negatives.
- **Solution:** An aggressive data augmentation pipeline (Mosaic, Mixup, HSV adjustments) was integrated during training. Furthermore, the operational confidence threshold was locked at exactly 40%. This strategically maximized the model's Recall capacity, ensuring that even ambiguous or motion-blurred hazards were reliably captured.

9.2. Hardware Integration & Environment Challenges

- **Challenge 3: GPS Signal Interruptions and Coordinate Inaccuracies**

- **Problem:** In environments with poor satellite visibility (indoors, emulators), the location service could enter an infinite loop, freezing the app (ANR). Even when a signal was caught, low-precision GPS data risked logging inaccurate hazard locations.
- **Solution:** A "High Accuracy" mandate was implemented alongside an asynchronous background listener waiting for the margin of error to drop. To prevent infinite loops, a 5-second Timeout Shield was integrated into the Geolocator service. If no precise location is found, a "Fallback" mechanism retrieves the device's "Last Known Position." Finally, users are provided with an interactive map (pin-drop) for manual verification before submission.

- **Challenge 4: Mobile-Server Communication Restrictions in Local Environments**

- **Problem:** During physical device testing, the Android operating system's security protocols blocked HTTP requests to the local Node.js server (localhost) due to cleartext traffic restrictions.
- **Solution:** The API configuration was transitioned from static addresses to a dynamic IP-based architecture. On the Android manifest, `networkSecurityConfig` and `usesCleartextTraffic` permissions were explicitly defined, granting controlled allowance for unencrypted data transfer over the local network. The backend was also configured to bind to all network interfaces (0.0.0.0).

9.3. Backend Storage, Security, and Event-Driven Routing

- **Challenge 5: High-Volume Media Ingestion & Database Latency**
 - **Problem:** Uploading high-resolution anomaly photos directly into the NoSQL database as Base64 strings caused severe query latency and threatened to rapidly exhaust cloud read/write billing quotas.
 - **Solution:** A Hybrid Storage Architecture was engineered. The Node.js gateway utilizes Multer middleware to intercept incoming Multipart/Form-Data. Instead of bloating the database, images are securely written to the server's local physical disk, generating a lightweight static URI. Only this URI and the relational metadata are persisted to Cloud Firestore, drastically optimizing the data load.
- **Challenge 6: Data Security and Payload Integrity in Multi-Role Architecture**
 - **Problem:** Preventing unauthorized data access between Citizens, Workers, and Admins (e.g., preventing a citizen from marking a task as "Resolved") and ensuring incoming data types were structurally sound.
 - **Solution:** A Zero-Trust gateway was established. Stateless user authentication is guaranteed through JWT validation, ensuring Role-Based Access Control (RBAC). Additionally, the integrity of incoming payload parameters is strictly validated using schemas defined by the Zod library, immediately rejecting malformed requests (e.g., string coordinates) before they reach the database layer.
- **Challenge 7: Instant Delivery of Urgent Notifications**
 - **Problem:** Critical infrastructure issues and emergency incidents require immediate attention. Without a dedicated prioritization mechanism, urgent reports could be delayed or become less visible among standard civic complaints, slowing response times and increasing operational risk.
 - **Solution:** To address this challenge, a dedicated emergency reporting workflow was integrated into the system. When a citizen submits a complaint through the Emergency Complaint Report module, the report is automatically tagged with a high-priority status. Upon reaching the database, event-driven triggers immediately highlight the record on the administrative dashboard and dispatch simultaneous push notifications marked as "Emergency" to the relevant municipal departments and field teams. This architecture ensures that emergency reports are instantly distinguished from standard complaints and receive rapid attention.

9.4. Localization and Concurrent Engineering

- **Challenge 8: Dynamic Content Management in a Multilingual Structure**
 - **Problem:** Ensuring language consistency for dynamic backend data and AI class labels when switching between Turkish and English. Additionally, Dart's `toUpperCase()` function corrupted Turkish characters (İ, Ğ, Ş), breaking search filters.

- **Solution:** Status updates and object classes are stored universally as Enums (e.g., class: pothole), which are translated in real-time on the frontend using local i18n dictionaries. For pure dynamic data, language-specific database fields (title_tr, title_en) were queried based on device locale. To resolve the filtering crashes, a custom Sanitization algorithm was developed to map Turkish characters to universal English equivalents.
- **Challenge 9: Version Control Risks and Sequential Integration**
 - **Problem:** Collaborating on a single monolithic codebase posed a severe risk of complex merge conflicts. It was recognized that simultaneous, parallel code pushes could potentially corrupt the production-ready state of the main branch, leading to significant project delays and broken UI states.
 - **Solution:** To strictly protect the integrity of the application, stability was prioritized over concurrent development speed through the implementation of a "Single Staging Branch" strategy. A dedicated v1 branch was established to isolate all active development from the core main branch. To entirely eliminate the risk of code collisions within v1, a synchronous (sequential) development protocol was adopted. Development schedules were intentionally coordinated; integration phases were strictly sequenced so that one module was fully completed and pushed to the repository before the next integration phase was initiated. Once the v1 environment was fully finalized and stabilized without any conflicts, it was safely merged into the main branch. This deliberate, controlled workflow successfully guaranteed zero structural breakages during the project lifecycle.

10. Conclusion & Future Work

10.1. Conclusion

This comprehensive graduation project successfully introduces an autonomous, secure, and end-to-end "Smart Municipality Application" that fundamentally modernizes urban framework maintenance. By transitioning the deep learning computational load to the edge via the YOLOv8 Nano architecture and TensorFlow Lite, the system completely eliminates cloud-based GPU hosting costs and strictly preserves citizen privacy through on-device data validation and instant background discarding.

The integration of a robust Node.js backend—fortified by JWT authentication, Zod payload validation, and a hybrid Multer-Firestore storage strategy—guarantees enterprise-grade security, preventing unauthorized multi-role access and ensuring zero-trust data integrity. On the client side, the Flutter-based single codebase successfully orchestrates role-based user interfaces (Citizens, Workers, Admins) while maintaining perfect UI fluidity through Dart Isolates and real-time synchronization via Firestore Streams. Ultimately, this engineered system successfully automates urban monitoring, shifting local government operations from a reactive posture to a proactive, data-driven strategy, and comprehensively bridges the communication gap between citizens and local governance.

10.2. Future Work and Vision

While the developed system possesses a robust infrastructure that successfully meets all core functional requirements and architectural objectives, future development phases will focus on rigorous performance testing and the integration of advanced modular capabilities:

- **System Testing and Algorithmic Optimization:** The accuracy rates and generalization capabilities of the edge AI model will be rigorously stress-tested under extreme lighting and adverse weather conditions. Furthermore, end-to-end data transmission payloads and background isolate performance will be continuously profiled to further minimize battery and network consumption.
- **Usability Testing (UX) and Field Deployment:** Comprehensive A/B testing and usability metrics will be gathered from distinct user groups (citizens and municipal personnel in the field). The user experience will be continuously refined based on empirical feedback loops.
- **Visual Proof of Resolution and Transparent Feedback:** To further enhance civic transparency and institutional accountability, a "Visual Verification" mechanism will be implemented for the final stage of the task lifecycle. In future iterations, municipal field workers will be strictly required to capture and upload a real-time photograph of the repaired infrastructure before the system permits them to mark a task as "Resolved". This photographic evidence will be seamlessly appended to the final database transaction and delivered directly to the reporting citizen's mobile interface alongside the resolution push notification, establishing a definitive, closed-loop trust mechanism.
- **Advanced Institutional Integrations:** Future development phases include integration with Türkiye's e-Devlet authentication infrastructure to provide secure user verification and strengthen accountability within the platform. In addition, Firebase Cloud Messaging (FCM) can be integrated to support scalable cross-platform push notifications for status updates, emergency alerts, and municipal announcements. To improve system reliability, Firebase Crashlytics may be employed for crash reporting and runtime error monitoring, enabling faster identification and resolution of issues in production environments.
- **Extensible Architecture (IoT & NLP):** The overarching system architecture has been intentionally engineered in a highly modular state. This flexible foundation allows for the seamless future integration of voice command processing (Natural Language Processing - NLP) for hands-free hazard reporting, as well as direct data pipelines from stationary IoT smart city sensors. These modules can be seamlessly appended in the future to expand the platform's autonomous capabilities without disrupting or damaging the existing core structure.

11. References

- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv preprint. <https://arxiv.org/abs/2004.10934>
- Chisnell, D., & Redish, J. C. (2005). *Designing Web Sites for Older Adults: A Review of the Literature and Expert Review of 50 Web Sites*. AARP. <https://danachisnell.com/wp-content/uploads/2025/04/AARP-50Sites.pdf>
- Çeliksoy, E., & Canga, H. D. (2024). Türkiye's technological change, e-government and public administration. *KMÜ Sosyal ve Ekonomik Araştırmalar Dergisi*, 26(47), 100–124. <https://easiv.kmu.edu.tr/server/api/core/bitstreams/4b1d436b-c0dd-4626-afa6-10f50d21d1c3/content>
- Firebase. (2026). *Cloud Firestore Realtime Updates [online]*. Firebase Documentation. <https://firebase.google.com/docs/firestore/query-data/listen>
- Google. (2026). *Flutter architectural overview*. Flutter Docs. <https://docs.flutter.dev/resources/architectural-overview>
- Google. (2026). *LiteRT: High-Performance On-Device Machine Learning Framework [online]*. Google AI Edge Docs. <https://developers.google.com/edge/litert>
- Maeda, H., Sekimoto, Y., Seto, T., Kashiwayama, T., & Omata, H. (2018). Road damage detection and classification using deep neural networks with smartphone images. *Computer-Aided Civil and Infrastructure Engineering*, 33(12), 1127–1141. https://www.researchgate.net/publication/326087983_Road_Damage_Detection_and_Classification_Using_Deep_Neural_Networks_with_Smartphone_Images_Road_damage_detection_and_classification
- Nielsen, J. (2020). 10 usability heuristics for user interface design. Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Node.js. (2026). Node.js v26.3.0 Documentation [online]. Node.js API Docs. <https://nodejs.org/docs/latest/api/>
- Offenhuber, D. (2015). Infrastructure legibility: A comparative analysis of open311-based citizen feedback systems. *Cambridge Journal of Regions, Economy and Society*, 8(1), 93–112. https://www.researchgate.net/publication/261912334_Infrastructure_Legibility-A_Comparative_Analysis_of_Open311-Based_Citizen_Feedback_Systems
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. arXiv preprint arXiv:1506.02640. <https://arxiv.org/abs/1506.02640>
- Ultralytics. (2023). *Home | Ultralytics Docs [online]*. Ultralytics Docs. <https://docs.ultralytics.com/>

Venkatesh, V., Morris, M. G., Davis, G. B., & Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. *MIS Quarterly*, 27(3), 425–478.

<https://www.researchgate.net/profile/Paul-Vossen-2/post/What-is-the-relevant-questionnaire-for-School-Liking-as-the-dependent-variable-among-students/attachment/5a4623de4cde266d587e8e22/AS%3A576862486056960%401514546142661/download/Venkatesh+User+Acceptance+of+Information+Technology+2003.pdf>

Wang, F., & Hannafin, M. J. (2005). Design-based research: Toward a methodological framework for educational research. *Educational Technology Research and Development*, 53(4), 5–23.

https://www.researchgate.net/publication/225626676_Design-based_research_and_technology-enhanced_learning_environments_Educational_Technology_Research_and_Development_534_5-23

Zod. (2026). Zod: TypeScript-first schema validation with static type inference [online]. Zod Documentation. <https://zod.dev/>